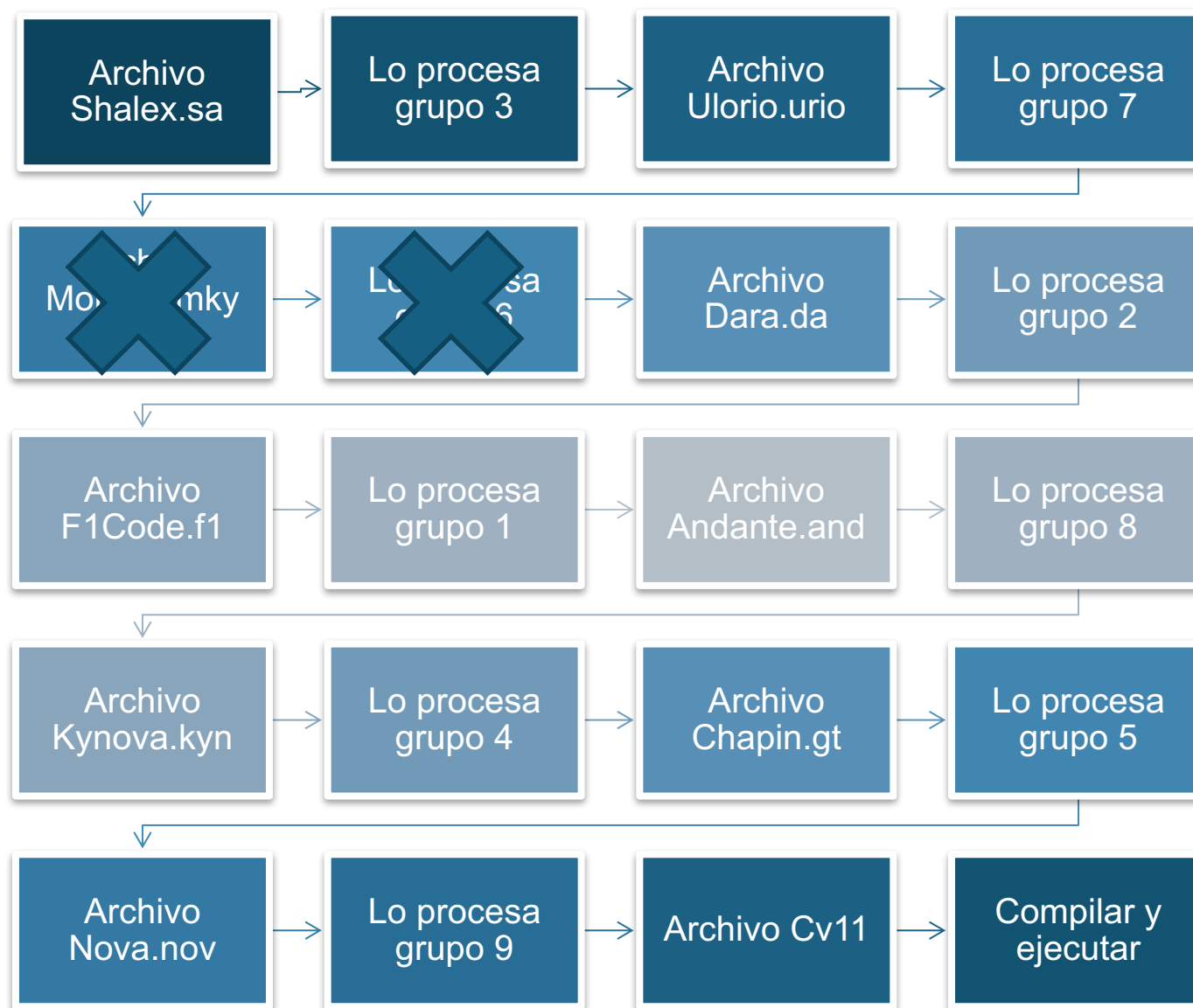


Curso Compiladores 3CT0113

Semana 10 – Derivaciones, ambigüedad, recursividad y factorización

Proyecto



Conexión con la semana 9

Temas vistos en la Semana 9

Repaso y Transición X

Temas

En la Semana 9 construimos los cimientos del análisis sintáctico:

- **Derivaciones:** Generación de cadenas paso a paso.
- **CST:** El Árbol de Sintaxis Concreta que captura la jerarquía.
- **Ambigüedad:** Cuando una gramática plana genera múltiples interpretaciones lógicas para una misma entrada.

¿Por qué eliminar la ambigüedad? Para que herramientas como **Bison** funcionen sin conflictos, debemos eliminar la ambigüedad. Esto se logra estratificando la gramática: encapsulando operaciones en diferentes niveles de precedencia para forzar un único árbol válido.

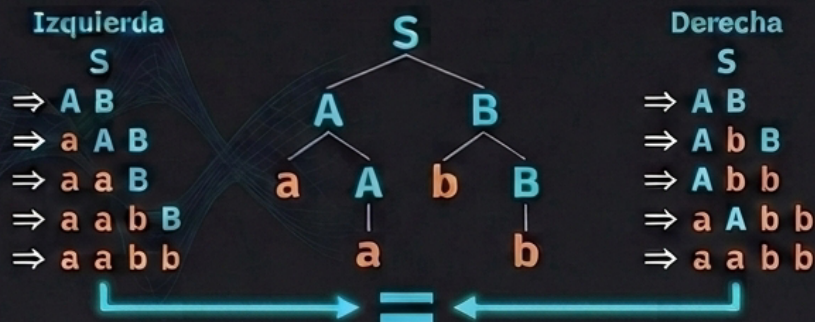
Panel Interactivo: Diagnóstico X

[Ambigüedad]

¿Dos derivaciones diferentes siempre significan que la gramática es ambigua?

[Derivaciones vs CST]

No. Una derivación por la izquierda (**Leftmost**) y una por la derecha (**Rightmost**) construyen la cadena en un orden distinto, generando secuencias de pasos diferentes. Sin embargo, si ambas convergen exactamente en el mismo CST, la gramática no es ambigua.



Conclusión: Ambas derivaciones construyen exactamente el mismo árbol sintáctico. La gramática no es ambigua. Una gramática solo es ambigua si produce dos árboles distintos para la misma cadena.

Notación para seguir los ejemplos

Notación	Significado
$\rightarrow$	Producción de la gramática
$\Rightarrow$	Un paso de derivación
$\Rightarrow^*$	Cero o más pasos
ε	Cadena vacía
i / n	Categorías didácticas: identificador / número

En $i=[E]+T$; los corchetes señalan la aparición elegida. No se escriben en la entrada.

Una producción por paso. Los terminales no se sustituyen.

Trabajo con las herramientas

Formal Language Editor

Elegir Context-Free Grammar. Reemplazar la regla inicial vacía.

Ingresa una alternativa por fila. Para ϵ , dejar vacío el lado derecho.

Seleccionar una hoja, elegir su producción y pulsar +step.

[https://www.cs.odu.edu/~zeil/automat/automat.c
gi](https://www.cs.odu.edu/~zeil/automat/automat.cgi)

CFG Developer

Ingresa las reglas en Create y comprobarlas en Verify.

Colocar una cadena por línea en Test y comparar los resultados.

Ignora espacios y mayúsculas/minúsculas: no usar la misma letra como terminal y no terminal.

<https://bakkot.github.io/cfgrammar-tool/>

D1 · Asignación: derivación izquierda

Paso	Producción	Forma resultante
0	Inicio	[S]
1	$S \rightarrow i=E;$	$i=[E];$
2	$E \rightarrow E+T$	$i=[E]+T;$
3	$E \rightarrow T$	$i=[T]+T;$
4	$T \rightarrow n$	$i=n+[T];$
5	$T \rightarrow n$	$i=n+n;$

$S \rightarrow i=E;$
 $E \rightarrow E+T \mid T$
 $T \rightarrow n$

Entrada objetivo: $i=n+n;$

En $i=T+T;$ se sustituye primero el T izquierdo.

Las dos apariciones de T son nodos distintos.

D1 · La misma entrada por la derecha

Paso	Producción	Forma resultante
0	Inicio	[S]
1	$S \rightarrow i=E;$	$i=[E];$
2	$E \rightarrow E+T$	$i=E+[T];$
3	$T \rightarrow n$	$i=[E]+n;$
4	$E \rightarrow T$	$i=[T]+n;$
5	$T \rightarrow n$	$i=n+n;$

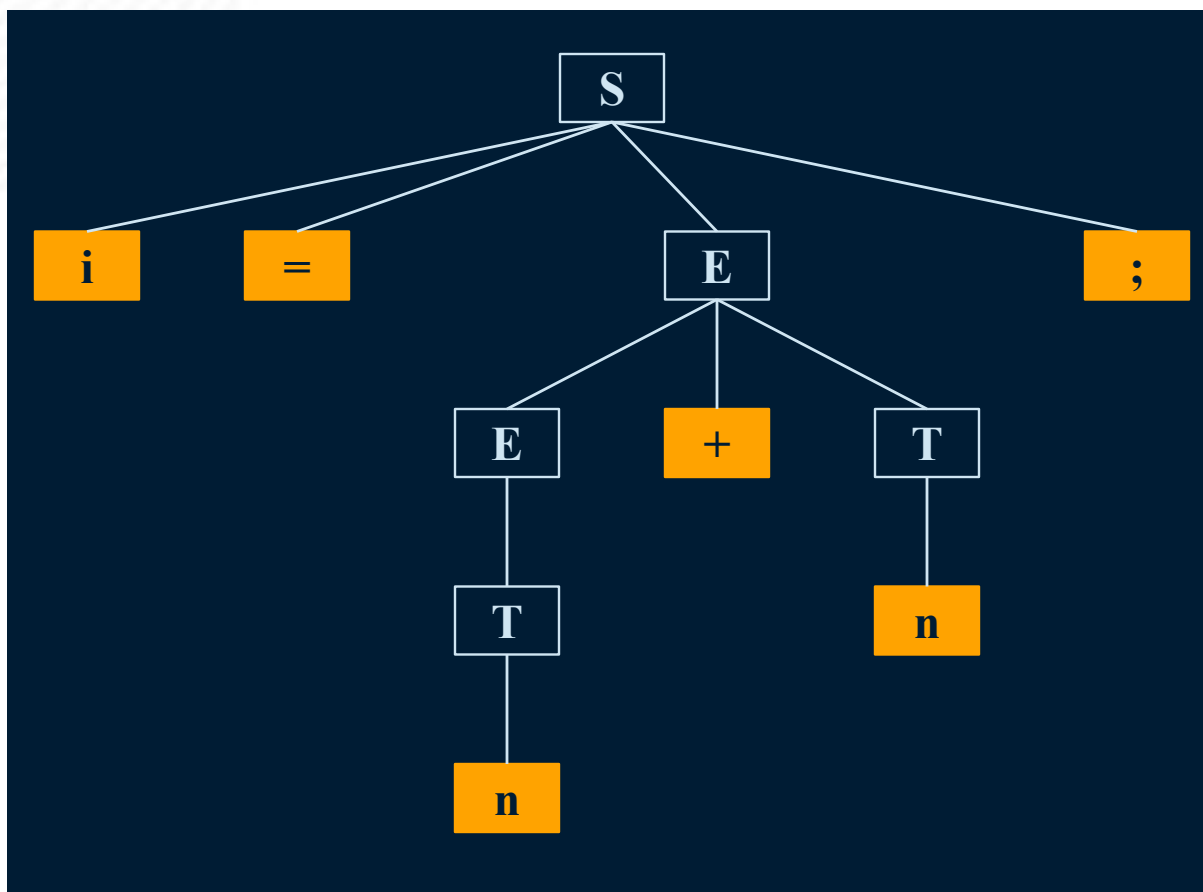
$S \rightarrow i=E;$
 $E \rightarrow E+T \mid T$
 $T \rightarrow n$

En $i=E+T$; se desarrolla T antes que E.

Cambian las formas intermedias, pero la entrada final es la misma.

Derivar por la derecha no significa escribir la cadena al revés.

D1 · CST



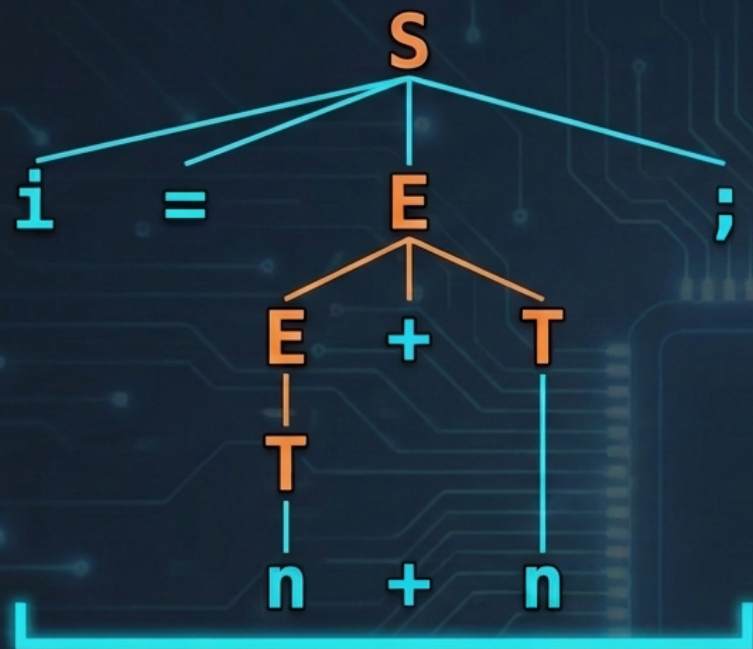
Los dos órdenes construyen este mismo CST.

Frontera (yield): $i=n+n;$

En Formal Language Editor, repetir ambos recorridos y comparar el árbol final.

Entrada	Resultado
$i=n;$	Aceptada
$i=n+n;$	Aceptada
$i=+n;$	Rechazada
$i=n+;$	Rechazada

D1 · Prueba en CFG Developer y Formal Language Tool



Frontera (Yield): $i=n+n;$

Pruebas en CFG Developer

Entrada	Verificación	Diagnóstico
$i=n;$	Aceptada	Deriva directamente vía $E \rightarrow T \rightarrow n$
$i=n+n;$	Aceptada	Asignación con suma completa válida
$i=+n;$	Rechazada	Falla sintáctica: falta primer operando
$i=n+;$	Rechazada	Falla sintáctica: falta segundo operando

Verificación en Formal Language Editor

Conclusión: El procedimiento demuestra empíricamente que tanto el recorrido por la izquierda como por la derecha derivan exactamente el mismo CST. El orden temporal de expansión no altera la jerarquía matemática ni estructural impuesta por las producciones.

D2 · Expresión con anidamiento

$\text{Expr} \rightarrow \text{Expr} + \text{Term} \mid \text{Term}$
 $\text{Term} \rightarrow \text{Term} * \text{Factor} \mid \text{Factor}$
 $\text{Factor} \rightarrow (\text{Expr}) \mid \text{id}$

Entrada objetivo: $i*(i+i)$

E organiza sumas.

T organiza productos.

F permite un identificador o una expresión entre paréntesis.

La suma se construye dentro del E que aparece al aplicar $F \rightarrow (E)$.

D2 · Izquierda: construcción exterior

Paso	Producción	Forma resultante
0	Inicio	[E]
1	$E \rightarrow T$	[T]
2	$T \rightarrow T * F$	[T]*F
3	$T \rightarrow F$	[F]*F
4	$F \rightarrow i$	$i*[F]$
5	$F \rightarrow (E)$	$i*([E])$
6	$E \rightarrow E + T$	$i*([E]+T)$

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid i$

Primero se completa el operando izquierdo del producto.

Después $F \rightarrow (E)$ abre una expresión interior.

D2 · Izquierda: completar la suma

Paso	Producción	Forma resultante
6	Continuación	$i*([E]+T)$
7	$E \rightarrow T$	$i*([T]+T)$
8	$T \rightarrow F$	$i*([F]+T)$
9	$F \rightarrow i$	$i*(i+[T])$
10	$T \rightarrow F$	$i*(i+[F])$
11	$F \rightarrow i$	$i*(i+i)$

$E \rightarrow E+T \mid T$
 $T \rightarrow T*F \mid F$
 $F \rightarrow (E) \mid i$

Se desarrolla siempre la aparición más a la izquierda.

Al finalizar, ya no queda ningún no terminal.

D2 · Derecha: construir el factor derecho

Paso	Producción	Forma resultante
0	Inicio	[E]
1	$E \rightarrow T$	[T]
2	$T \rightarrow T * F$	$T * [F]$
3	$F \rightarrow (E)$	$T * ([E])$
4	$E \rightarrow E + T$	$T * (E + [T])$
5	$T \rightarrow F$	$T * (E + [F])$
6	$F \rightarrow i$	$T * ([E] + i)$

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid i$

El F derecho del producto se desarrolla antes que el T izquierdo.

Todavía no se genera el primer i de la entrada.

D2 · Derecha: completar el recorrido

Paso	Producción	Forma resultante
6	Continuación	$T*([E]+i)$
7	$E \rightarrow T$	$T*([T]+i)$
8	$T \rightarrow F$	$T*([F]+i)$
9	$F \rightarrow i$	$[T]*(i+i)$
10	$T \rightarrow F$	$[F]*(i+i)$
11	$F \rightarrow i$	$i*(i+i)$

$E \rightarrow E+T \mid T$
 $T \rightarrow T*F \mid F$
 $F \rightarrow (E) \mid i$

La expresión interior termina antes que el primer operando.

Ambos recorridos llegan a $i*(i+i)$.

D2 · Prueba en CFG Developer y Formal Language Editor

Pruebas en CFG Developer y Formal Language Editor

CFG Developer

Grammar Input

```
S → E
E → T | E + T
T → F | T * F
F → ( E ) | i
```

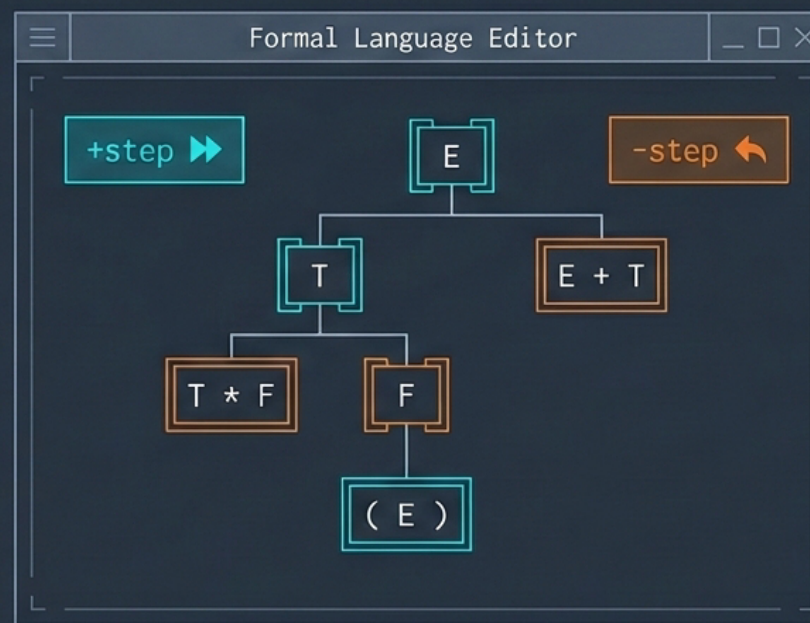
Test String

i*(i+i)

Output Table

String	Result
i*(i+i)	Matches: Yes ✓

CFG Developer: Valida pertenencia. Escribimos la gramática (lado derecho tras la flecha), ingresamos cadenas por línea en Test, y observamos si la tabla indica **Matches: Yes**. Distingue mayúsculas/minúsculas.



Formal Language Editor: Construye el CST manualmente. Seleccionamos una hoja No Terminal, elegimos la regla, y usamos **+step** para expandir (o **-step** para corregir).

D3 · Un bloque con dos sentencias

$$\begin{aligned} B &\rightarrow \{L\} \\ L &\rightarrow SL \mid S \\ S &\rightarrow i=E; \\ E &\rightarrow E+n \mid n \end{aligned}$$

Entrada: $\{i=n; i=n+n;\}$

B representa el bloque.

L representa una lista no vacía.

S representa una asignación.

$L \rightarrow SL$ continúa la lista. $L \rightarrow S$ la termina.

D3 · Derivación izquierda completa

Paso	Producción	Forma resultante
0	Inicio	[B]
1	$B \rightarrow \{L\}$	$\{[L]\}$
2	$L \rightarrow SL$	$\{[S]L\}$
3	$S \rightarrow i=E;$	$\{i=[E];L\}$
4	$E \rightarrow n$	$\{i=n;[L]\}$
5	$L \rightarrow S$	$\{i=n;[S]\}$
6	$S \rightarrow i=E;$	$\{i=n;i=[E];\}$
7	$E \rightarrow E+n$	$\{i=n;i=[E]+n;\}$
8	$E \rightarrow n$	$\{i=n;i=n+n;\}$

$B \rightarrow \{L\}$
 $L \rightarrow SL \mid S$
 $S \rightarrow i=E;$
 $E \rightarrow E+n \mid n$

Se construye primero la primera sentencia.

Conservar todas las llaves, los signos = y los puntos y coma.

Nunca sustituir dos apariciones de S o de E en un mismo paso.

D3 · Derivación derecha completa

Paso	Producción	Forma resultante
0	Inicio	[B]
1	$B \rightarrow \{L\}$	$\{[L]\}$
2	$L \rightarrow SL$	$\{S[L]\}$
3	$L \rightarrow S$	$\{S[S]\}$
4	$S \rightarrow i=E;$	$\{Si=[E];\}$
5	$E \rightarrow E+n$	$\{Si=[E]+n;\}$
6	$E \rightarrow n$	$\{[S]i=n+n;\}$
7	$S \rightarrow i=E;$	$\{i=[E];i=n+n;\}$
8	$E \rightarrow n$	$\{i=n;i=n+n;\}$

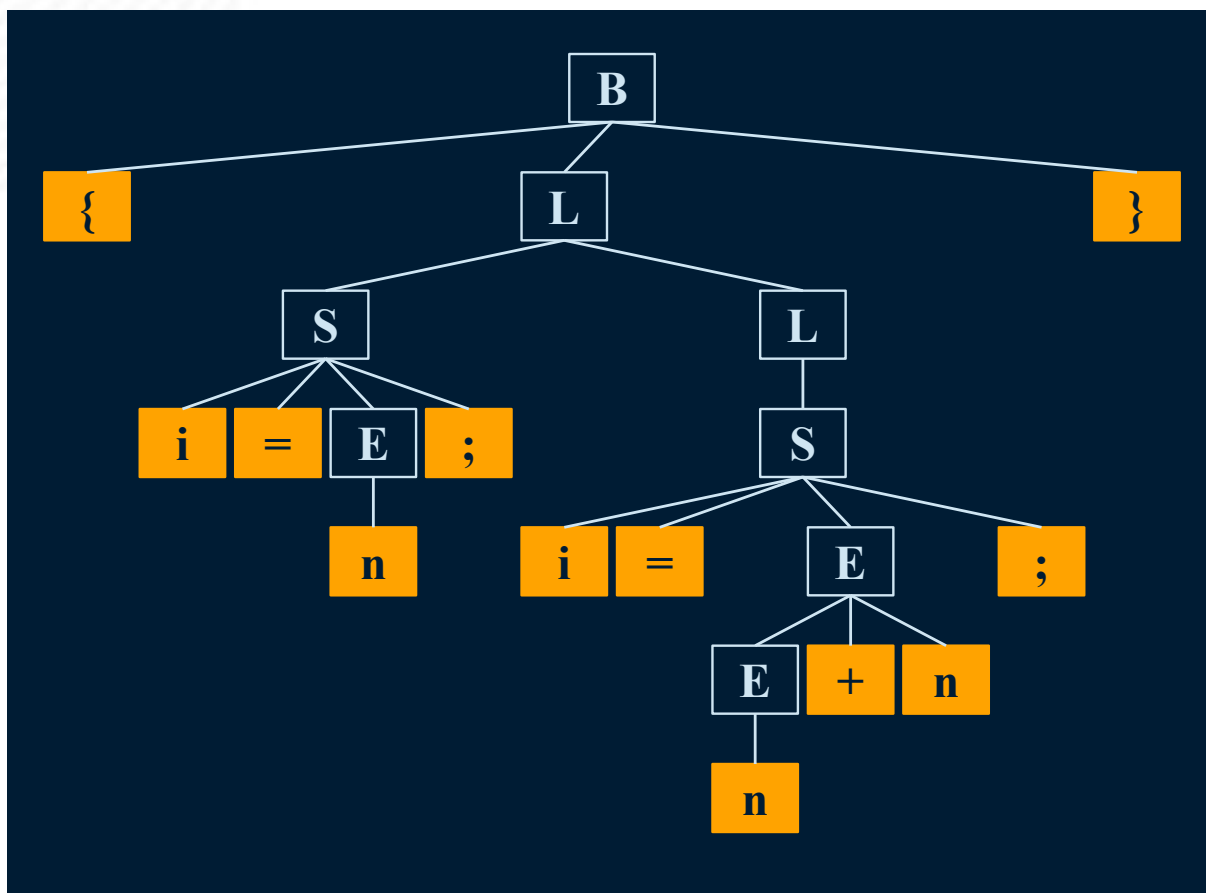
$B \rightarrow \{L\}$
 $L \rightarrow SL \mid S$
 $S \rightarrow i=E;$
 $E \rightarrow E+n \mid n$

En $\{SS\}$, elegir la segunda aparición.

La segunda sentencia se construye antes que la primera.

La cadena final conserva el orden original de las sentencias.

D3 · Árbol y resultados



Frontera (yield): {i=n;i=n+n;}

En Formal Language Editor, comenzar en B y repetir los dos recorridos.

Entrada	Resultado
{i=n;}	Aceptada
{i=n;i=n+n;}	Aceptada
{}	Rechazada
{i=n;i=;}	Rechazada

El bloque vacío se rechaza porque no existe $L \rightarrow \epsilon$.

Eliminación de ambigüedad

¿Qué significa realmente eliminar la ambigüedad? (El Problema de la Topología)

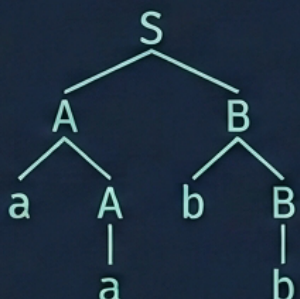
Una Gramática Libre de Contexto (GLC) es ambigua si y solo si existe al menos una cadena en su lenguaje que puede ser generada por dos Árboles de Análisis Sintáctico (CST) distintos.

Orden Temporal $\neq$ Ambigüedad

SEGURO

Derivación por
la Izquierda
(Leftmost)

S $\rightarrow$
 $\Rightarrow$ AB $\rightarrow$
 $\Rightarrow$ aB $\rightarrow$
 $\Rightarrow$ aaB $\rightarrow$
 $\Rightarrow$ aaa $\rightarrow$
 $\Rightarrow$ aabb $\rightarrow$



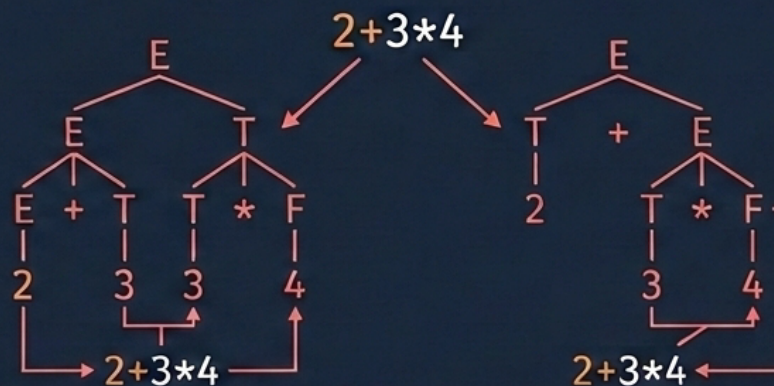
Derivación por
la derecha
(Rightmost)

S $\leftarrow$
 $\Rightarrow$ AB $\leftarrow$
 $\Rightarrow$ A(bB) $\leftarrow$
 $\Rightarrow$ A(bb) $\leftarrow$
 $\Rightarrow$ [A]bb $\leftarrow$
 $\Rightarrow$ aabb $\leftarrow$

Derivar por la izquierda o por la derecha produce secuencias distintas de pasos, pero convergen en el mismo árbol.

Topología Alternativa = Ambigüedad Matemática

PELIGRO



Resultado: 20
(Asociatividad por la izquierda/Beja precedencia)

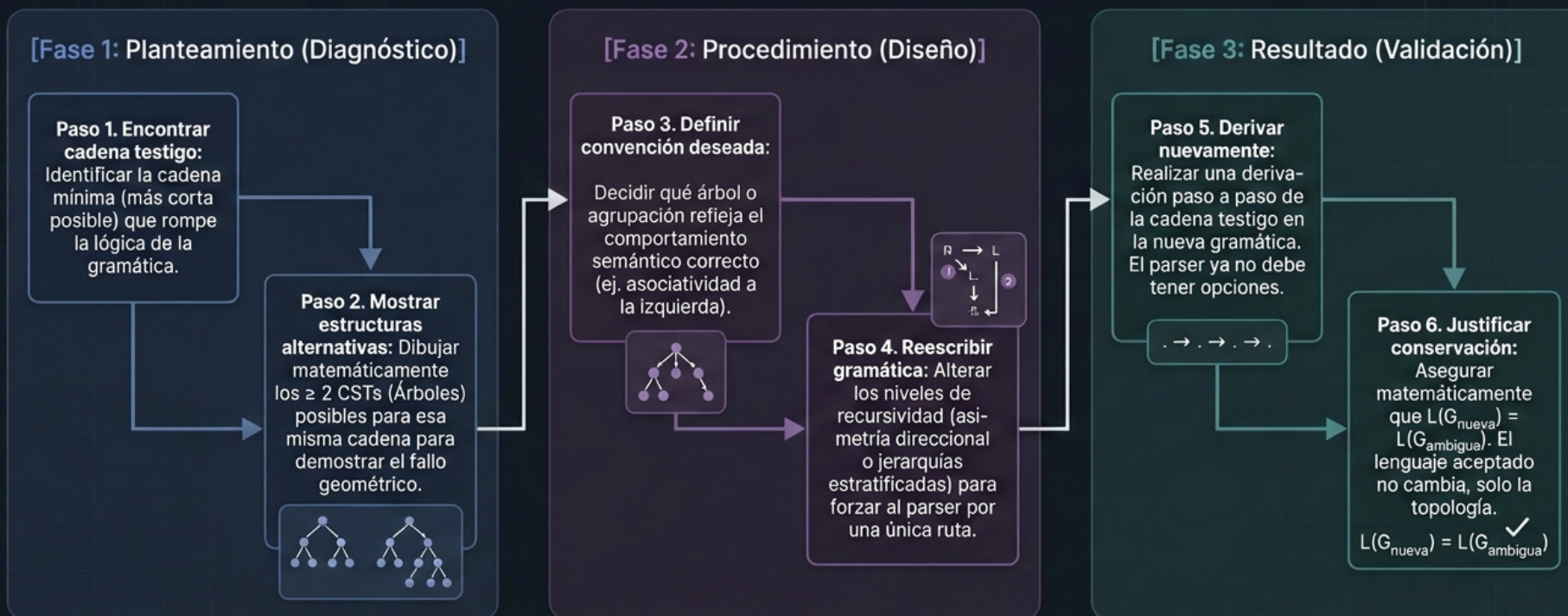
Resultado: 14
(Asociatividad por la derecha/Mayor precedencia)

Una misma cadena exacta genera agrupaciones jerárquicas totalmente diferentes.

Para eliminar la ambigüedad, no se debe dejar al parser elegir libremente. Hay que elegir una convención (asociatividad/precedencia) y codificarla rígidamente en las producciones.

Procedimiento para desambiguar

Método de trabajo: Protocolo de reingeniería de gramáticas

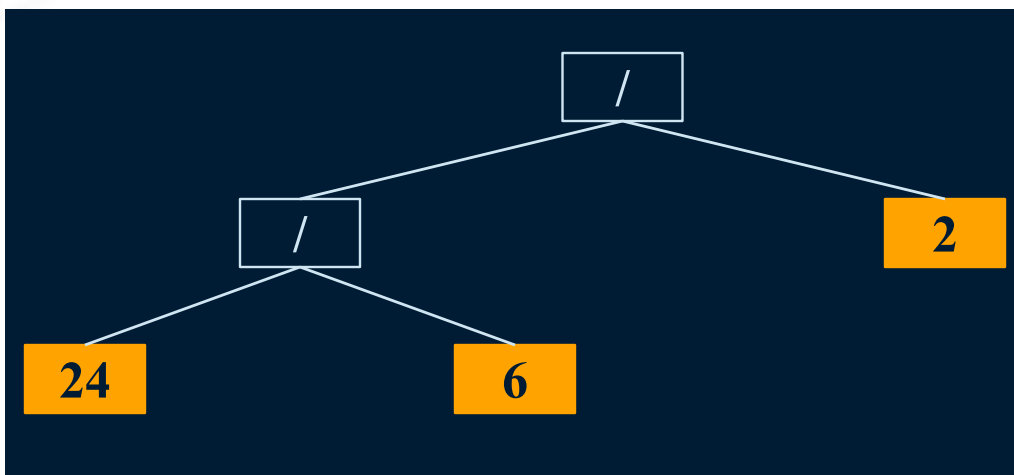


⚠ Nota: Existen lenguajes inherentemente ambiguos donde no existe ninguna gramática reparada posible. En programación, evitamos deliberadamente crear estas estructuras.

A1 · Dos interpretaciones de una división

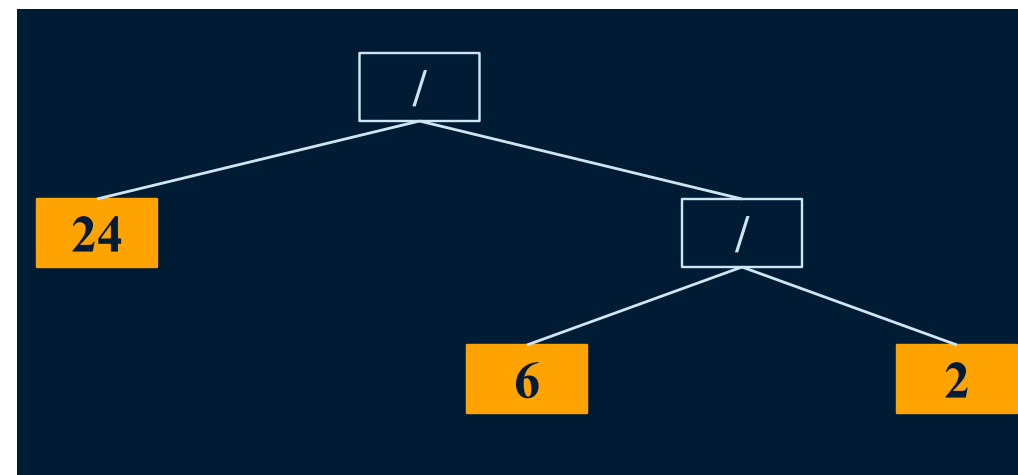
$$E \rightarrow E/E \mid n$$

$$24/6/2 = 2$$



$$\begin{aligned} E &\Rightarrow E/E \Rightarrow E/E/E \Rightarrow n/E/E \\ &\Rightarrow n/n/E \Rightarrow n/n/n \end{aligned}$$

$$24/6/2 = 8$$



$$\begin{aligned} E &\Rightarrow E/E \Rightarrow n/E \Rightarrow n/E/E \\ &\Rightarrow n/n/E \Rightarrow n/n/n \end{aligned}$$

Misma cadena n/n/n. Dos derivaciones izquierdas y dos agrupaciones distintas.

A1 · Asociatividad izquierda en las reglas

$$\begin{aligned} E &\rightarrow E/T \mid T \\ T &\rightarrow n \end{aligned}$$

Convención: las divisiones consecutivas se agrupan de izquierda a derecha.

E conserva las divisiones previas en la rama izquierda.

T limita el operando derecho a un número. No puede desarrollar otra división.

La gramática expresa la convención, en lugar de dejarla implícita.

A1 · Comprobación de la reparación

E
⇒ E/T
⇒ E/T/T
⇒ T/T/T
⇒ n/T/T
⇒ n/n/T
⇒ n/n/n

La división exterior es la última /.

Agrupación: $(24/6)/2 = 2$.

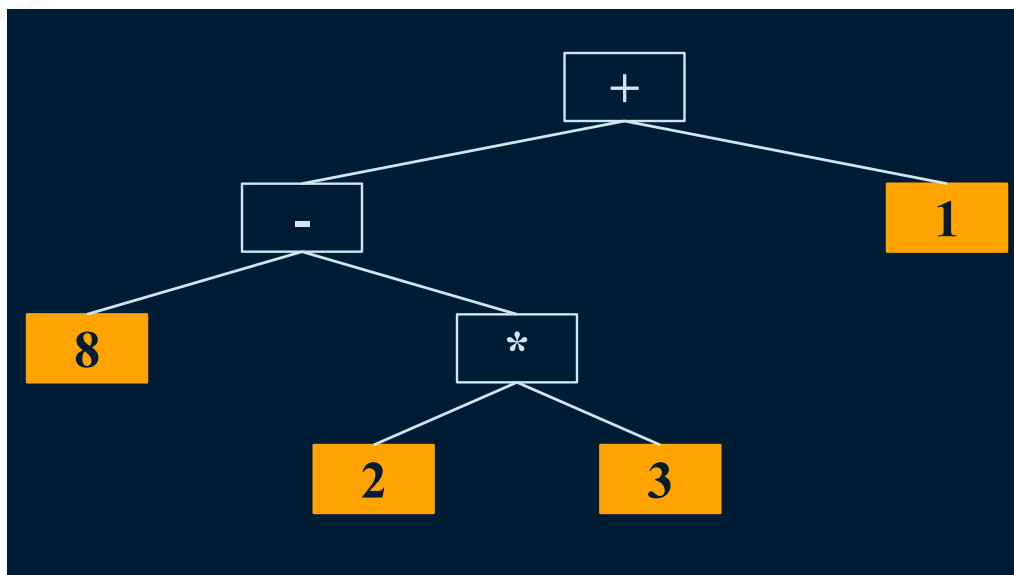
Ambas gramáticas generan n seguido de cero o más repeticiones de /n.

Variante derecha: $E \rightarrow T/E \mid T$, con $T \rightarrow n$.

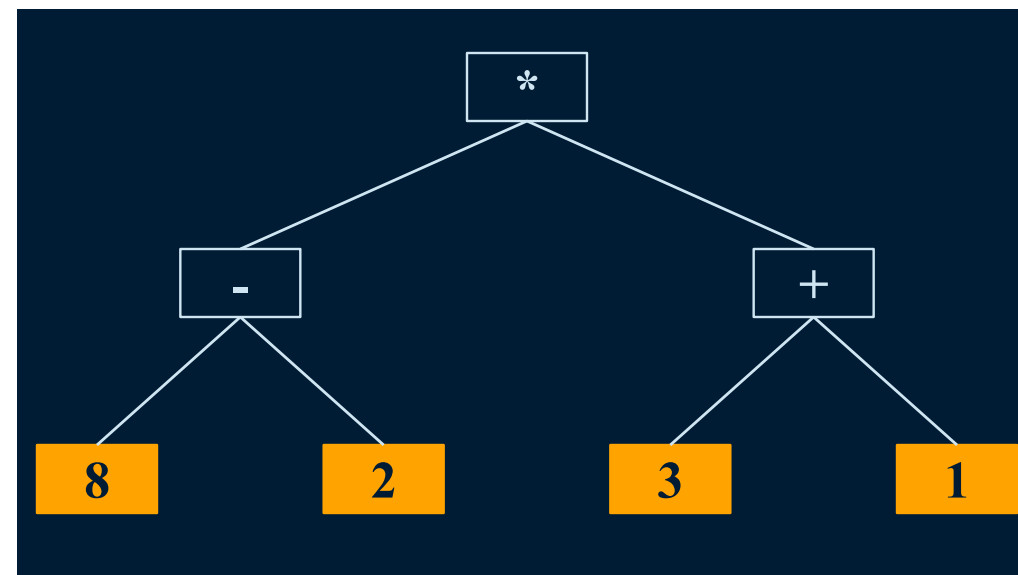
A2 · Varios operadores, dos estructuras

$$E \rightarrow E+E \mid E-E \mid E * E \mid (E) \mid n$$

$$8-2*3+1 = 3$$



$$8-2*3+1 = 24$$



Entrada original: $8-2*3+1$. La CFG inicial permite estas dos agrupaciones.

A2 · Niveles sintácticos E, T y F

E

$$E \rightarrow E+T \mid E-T \mid T$$

Suma y resta: asociación izquierda

T

$$T \rightarrow T * F \mid F$$

Multiplicación: nivel más interno

F

$$F \rightarrow (E) \mid n$$

Número o expresión agrupada

A2 · Derivación reparada: primera parte

E
 $\Rightarrow E+T$
 $\Rightarrow E-T+T$
 $\Rightarrow T-T+T$
 $\Rightarrow F-T+T$
 $\Rightarrow n-T+T$
 $\Rightarrow n-T*F+T$
 $\Rightarrow n-F*F+T$
 $\Rightarrow n-n*F+T$
 $\Rightarrow n-n*n+T$
 $\Rightarrow n-n*n+F$
 $\Rightarrow n-n*n+n$

Objetivo: $n-n*n+n$.

La multiplicación aparece dentro del T que es operando de la resta.

$$E \rightarrow E+T \mid E-T \mid T$$

$$T \rightarrow T*F \mid F$$

$$F \rightarrow (E) \mid n$$

A3 · El else pendiente

Else interior

```
if b then  
  [if b then a else a]
```

Else exterior

```
if b then  
  [if b then a]  
else a
```

$S \rightarrow ibtS \mid ibtSeS \mid a$

Entrada: ibtibtaea

i = if, b = condición, t = then, e = else, a = sentencia simple.

¿A cuál if pertenece el único else?

A3 · Sentencias emparejadas y pendientes

$$\begin{aligned} S &\rightarrow M \mid U \\ M &\rightarrow \text{ibtMeM} \mid a \\ U &\rightarrow \text{ibtS} \mid \text{ibtMeU} \end{aligned}$$

$$S \rightarrow \text{ibtS} \mid \text{ibtSeS} \mid a$$

M: todos sus if están emparejados. La sentencia simple a también pertenece a M.

U: conserva algún if pendiente.

Antes de un else explícito se exige M, para que no quede un if interno que lo reclame.

A3 · Recorrido de la gramática reparada

S
⇒ U
⇒ ibtS
⇒ ibtM
⇒ ibtibMeM
⇒ ibtibtaeM
⇒ ibtibtaea

El if exterior queda pendiente.

El if interior se deriva desde M e incluye su else.

$S \rightarrow M \mid U$
 $M \rightarrow ibtMeM \mid a$
 $U \rightarrow ibtS \mid ibtMeU$

El else pertenece a la sentencia interior.

A3 · Justificación y verificación

Entrada	Resultado
a	Aceptada
ibta	Aceptada
ibtaea	Aceptada
ibtibtaea	Aceptada
ibtea	Rechazada

Si el else fuera exterior, ibta tendría que generarse desde M. Eso es imposible: ibta contiene un if sin else.

Comparar en el editor los dos árboles originales y el árbol reparado.

Recursividad izquierda

$A \Rightarrow^+ A\alpha$
(uno o más pasos)

Ejemplo directo:

$E \rightarrow E+n \mid n$

Directa: aparece en una producción.

Indirecta: aparece después de pasar por otros no terminales.

Un parser descendente predictivo básico puede volver a llamar a E sin consumir entrada.

Bison admite recursividad izquierda. Eliminarla no es un requisito para usarlo.

Transformación de la recursividad directa

Antes

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \\ \mid \beta_1 \mid \beta_2$$

Después

$$A \rightarrow \beta_1 R \mid \beta_2 R \\ R \rightarrow \alpha_1 R \mid \alpha_2 R \mid \varepsilon$$

α : sufijos no vacíos de las alternativas recursivas. β : bases productivas que no comienzan por A.

R es nuevo. ε permite detener las continuaciones.

Separar bases y sufijos antes de escribir las reglas nuevas.

R1 · Transformación de una suma

Recursividad directa

$$E \rightarrow E+n \mid n$$

$$\alpha = +n$$

$$\beta = n$$

Base y continuaciones

$$E \rightarrow nR$$

$$R \rightarrow +nR \mid \varepsilon$$

Primero se genera n . Después se añaden cero o más continuaciones $+n$.

R1 · Derivaciones comparadas

Antes

$$\begin{aligned} E \\ \Rightarrow E+n \\ \Rightarrow E+n+n \\ \Rightarrow n+n+n \end{aligned}$$

$$E \rightarrow E+n \mid n$$

Después

$$\begin{aligned} E \\ \Rightarrow nR \\ \Rightarrow n+nR \\ \Rightarrow n+n+nR \\ \Rightarrow n+n+n \end{aligned}$$

$$\begin{aligned} E &\rightarrow nR \\ R &\rightarrow +nR \mid \varepsilon \end{aligned}$$

Caso base: $E \Rightarrow nR \Rightarrow n$. Sin $R \rightarrow \varepsilon$, ninguna repetición puede terminar.

La cadena se conserva. La forma del CST cambia.

R2 · Varias bases y varios sufijos

Lista no vacía

$$L \rightarrow L,i \mid L,n \mid i \mid n$$

$$\alpha_1 = ,i \quad \alpha_2 = ,n$$

$$\beta_1 = i \quad \beta_2 = n$$

Gramática transformada

$$L \rightarrow iR \mid nR$$

$$R \rightarrow ,iR \mid ,nR \mid \varepsilon$$

Cada alternativa base debe conservarse. Cada sufijo recursivo debe pasar a R.

Olvidar nR eliminaría todas las listas que empiezan por un número.

R2 · Lista completa y pruebas

Antes

$$\begin{aligned} L \\ \Rightarrow L,i \\ \Rightarrow L,n,i \\ \Rightarrow i,n,I \end{aligned}$$

$$L \rightarrow L,i \mid L,n \mid i \mid n$$

Después

$$\begin{aligned} L \\ \Rightarrow iR \\ \Rightarrow i,nR \\ \Rightarrow i,n,iR \\ \Rightarrow i,n,I \end{aligned}$$

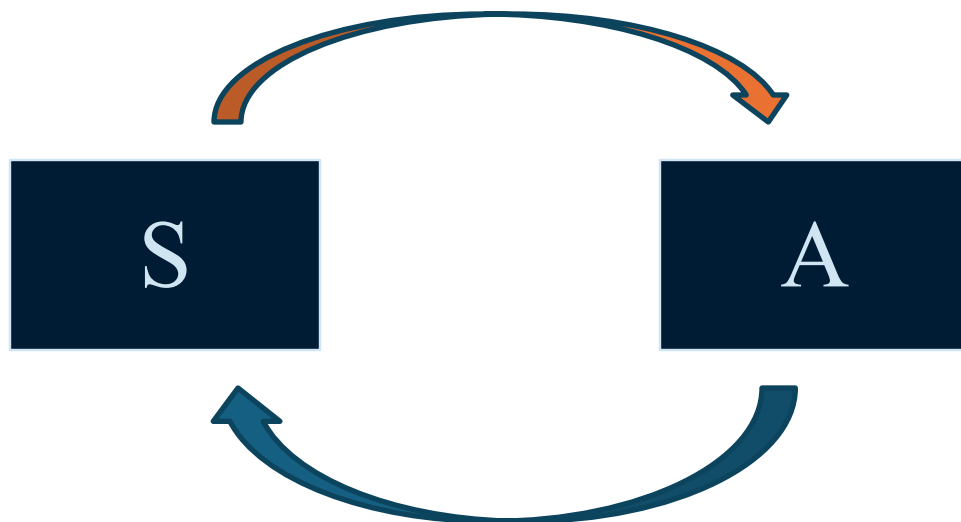
$$\begin{aligned} L &\rightarrow iR \mid nR \\ R &\rightarrow ,iR \mid ,nR \mid \varepsilon \end{aligned}$$

También: $L \Rightarrow nR \Rightarrow n,iR \Rightarrow n,i$.

Aceptadas: «i», «n», «i,n,i». Rechazadas: «,i» e «i,,n».

R3 · Recursividad indirecta

$$S \rightarrow Aa \mid b$$

$$A \rightarrow Sc \mid d$$


Ninguna producción comienza directamente por su propio no terminal.

$S \Rightarrow Aa \Rightarrow Sca$: S reaparece al inicio después de dos pasos.

El ciclo $S \rightarrow A \rightarrow S$ representa dependencias iniciales, no un CST.

R3 · Sustitución que revela el ciclo

Expandir S en $A \rightarrow Sc$

$S \rightarrow Aa$ aporta Aac
 $S \rightarrow b$ aporta bc

Resultado

$A \rightarrow Aac \mid bc \mid d$

La c final se conserva en cada sustitución.

Ahora $A \rightarrow Aac$ hace visible la recursividad directa.

$A \rightarrow Aa \mid b \mid d$ sería incorrecto: pierde el sufijo c.

R3 · Eliminación de la recursividad directa

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow Sc \mid d \end{aligned}$$
$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow bcR \mid dR \\ R &\rightarrow acR \mid \varepsilon \end{aligned}$$

$\alpha = ac; \beta_1 = bc; \beta_2 = d.$

A comienza ahora por b o d, no por S ni por A.

El orden del sufijo importa: α es ac, no ca.

El ciclo de llamadas iniciales ha desaparecido.

R3 · Derivación y conservación del lenguaje

Antes

$$\begin{aligned} S &\Rightarrow Aa \Rightarrow Sca \\ &\Rightarrow Aaca \Rightarrow Scaca \\ &\Rightarrow bcaca \end{aligned}$$

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow Sc \mid d \end{aligned}$$

Después

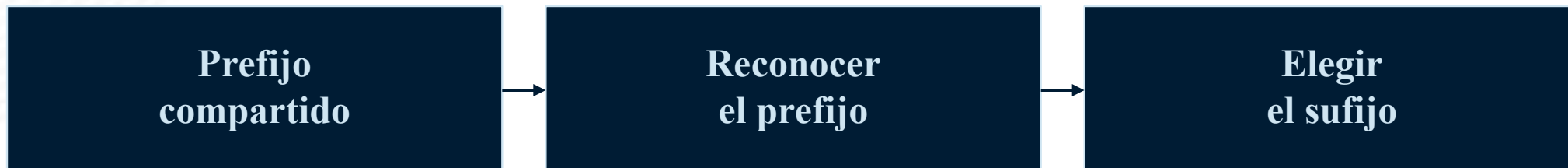
$$\begin{aligned} S &\Rightarrow Aa \\ &\Rightarrow bcRa \\ &\Rightarrow bcacRa \\ &\Rightarrow bcaca \end{aligned}$$

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow bcR \mid dR \\ R &\rightarrow acR \mid \varepsilon \end{aligned}$$

A genera una base bc o d seguida de cero o más repeticiones de ac.

Aceptadas: b, da, bca, daca, bcaca. Rechazadas: ba, dca.

Factorización izquierda



Varias alternativas pueden comenzar con la misma secuencia.

La factorización permite posponer la elección hasta después del prefijo común.

Es distinta de la recursividad izquierda y de la ambigüedad.

Fórmula de factorización

Antes

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \gamma$$

α es el prefijo común no vacío. γ queda fuera del grupo.

Si una alternativa termina al acabar α , su sufijo es ε .

Después

$$\begin{aligned} A &\rightarrow \alpha R \mid \gamma \\ R &\rightarrow \beta_1 \mid \beta_2 \end{aligned}$$

F1 · Asignación o llamada

Mismo comienzo

$S \rightarrow i=n; \mid i();$

Prefijo: i

Sufijos: =n; y ();

Decisión posterior

$S \rightarrow iR$

$R \rightarrow =n; \mid ();$

Después de i, el token = identifica la asignación y (identifica la llamada.

F1 · Reconstrucción de las alternativas

Asignación

$$S \Rightarrow iR \Rightarrow i=n;$$

$$S \rightarrow i=n; \mid i();$$

Llamada

$$S \Rightarrow iR \Rightarrow i();$$

$$S \rightarrow iR$$

$$R \rightarrow =n; \mid ();$$

$i;$ e $i(n);$ se rechazan porque no existe un sufijo correspondiente.

La gramática inicial no era ambigua: estas alternativas generan cadenas distintas.

Factorizar puede ser útil aunque no exista ambigüedad.

F2 · Primer nivel de factorización

Gramática inicial

$$S \rightarrow abc \mid abd \mid ae \\ \mid a \mid f$$

Extraer a

$$S \rightarrow aR \mid f \\ R \rightarrow bc \mid bd \mid e \mid \varepsilon$$

Las primeras cuatro alternativas comienzan por a.

Sus sufijos son bc, bd, e y ε . La alternativa f se conserva fuera del grupo.

F2 · Segundo nivel de factorización

$$\begin{aligned} S &\rightarrow aR \mid f \\ R &\rightarrow bc \mid bd \mid e \mid \varepsilon \end{aligned}$$
$$\begin{aligned} S &\rightarrow aR \mid f \\ R &\rightarrow bT \mid e \mid \varepsilon \\ T &\rightarrow c \mid d \end{aligned}$$

Dentro de R, bc y bd todavía comparten b.

El nuevo T decide entre c y d.

No agregar $T \rightarrow \varepsilon$: eso introduciría la cadena ab.

Cada no terminal nuevo contiene solo los sufijos que realmente existían.

F2 · Comprobación de todo el lenguaje

Cadena	Derivación
abc	$S \Rightarrow aR \Rightarrow abT \Rightarrow abc$
abd	$S \Rightarrow aR \Rightarrow abT \Rightarrow abd$
ae	$S \Rightarrow aR \Rightarrow ae$
a	$S \Rightarrow aR \Rightarrow a$
f	$S \Rightarrow f$

Lenguaje completo: {abc, abd, ae, a, f}. Las cadenas ab y af se rechazan.

F3 · Factorización del if

Antes

$$S \rightarrow ibtS \mid ibtSeS \mid a$$

Después

$$S \rightarrow ibtSR \mid a$$

$$R \rightarrow eS \mid \varepsilon$$

Prefijo común: $ibtS$. Sufijos: ε y eS .

R expresa presencia o ausencia de e .

¿Esta transformación ya obliga a unir e al if más cercano?

F3 · Dos derivaciones izquierdas

Else interior

S
 $\Rightarrow$ ibtSR
 $\Rightarrow$ ibtibtSRR
 $\Rightarrow$ ibtibtaRR
 $\Rightarrow$ ibtibtaeSR
 $\Rightarrow$ ibtibtaeaR
 $\Rightarrow$ ibtibtaea

$S \rightarrow \text{ibtSR} \mid a$
 $R \rightarrow \text{eS} \mid \varepsilon$

Else exterior

S
 $\Rightarrow$ ibtSR
 $\Rightarrow$ ibtibtSRR
 $\Rightarrow$ ibtibtaRR
 $\Rightarrow$ ibtibtaR
 $\Rightarrow$ ibtibtaeS
 $\Rightarrow$ ibtibtaea

En ibtibtaRR, cada R pertenece a un if distinto. La misma cadena conserva dos estructuras.

La gramática factorizada todavía es ambigua.

F3 · Alcance de la transformación

Lo que se resolvió

Se extrajo el prefijo compartido.

Se pospuso la elección entre los sufijos.

Lo que falta definir

A cuál if pertenece else.

La separación M/U de A3 expresa esa convención.

Selección de la técnica

Problema	Técnica	Ejemplo
Una cadena, dos árboles	Desambiguación	A1
No terminal que reaparece al inicio	Eliminar recursividad si el método lo requiere	R3
Alternativas con prefijo común	Factorización izquierda	F2
Conflictos entre operadores	Precedencia y asociatividad	B3

Identificar el problema antes de transformar la gramática o configurar el parser.

Alcance de las comprobaciones

Evidencia	Qué permite concluir
Una derivación completa	La cadena pertenece al lenguaje
Dos CST distintos para una cadena	La gramática es ambigua
Pruebas coincidentes antes/después	Evidencia útil, no equivalencia general
Bison sin advertencias	No demuestra por sí solo una CFG no ambigua

Para justificar una transformación, explicar cómo reconstruye las mismas bases, continuaciones o clases de estructuras.

Precedencia y asociatividad en Bison

```
%left '+' '-'  
%left '*' '/'
```

Cada línea representa un nivel. Las líneas posteriores tienen mayor precedencia.

%left: asociación izquierda. %right: asociación derecha.

%nonassoc: error ante un encadenamiento conflictivo del mismo nivel.

Regla y lookahead: la decisión

Situación del conflicto shift/reduce	Decisión
Lookahead con mayor precedencia	Shift
Regla con mayor precedencia	Reduce
Igual nivel con %left	Reduce
Igual nivel con %right	Shift
Igual nivel con %nonassoc	Error

La regla toma normalmente la precedencia de su último terminal. %prec puede modificarla.

B1 · Contrato y niveles de precedencia

```
%union { double numero; }  
%token <numero> NUM  
%type <numero> expr
```

```
%left '+' '-'  
%left '*' '/'
```

Entrada: 18-6/3-2

NUM transporta los valores. expr también tiene valor numérico.

Multiplicación y división superan a suma y resta. Todos estos niveles se asocian a la izquierda.

B1 · Reglas y acciones semánticas

```
expr:
  NUM          { $$ = $1; }
| '(' expr ')' { $$ = $2; }
| expr '+' expr { $$ = $1 + $3; }
| expr '-' expr { $$ = $1 - $3; }
| expr '*' expr { $$ = $1 * $3; }
| expr '/' expr { $$ = $1 / $3; }
;
```

\$1 y \$3 son operandos; \$\$ es el resultado. En (expr), el valor está en \$2.

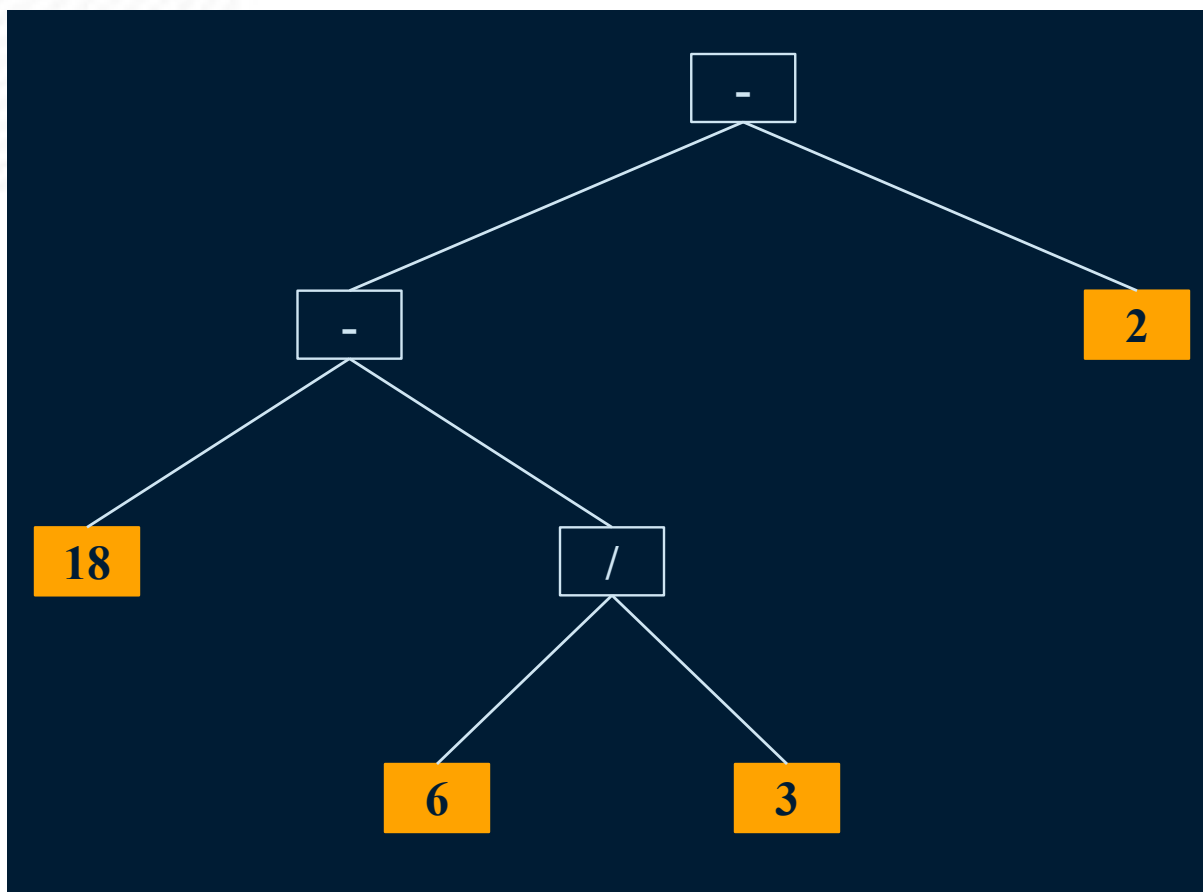
Estas acciones calculan valores. No construyen un AST.

B1 · Decisiones paso a paso

Situación conceptual	Decisión y consecuencia
18-6 con / como lookahead	Shift: / tiene mayor precedencia
6/3 con - como lookahead	Reduce: $6/3 = 2$
18-2 con otro -	Reduce por %left: $18-2 = 16$
16-2 al final	Completar la expresión: 14

Se muestran las decisiones relevantes, no una traza exhaustiva de estados LR.

B1 · Resultado y variantes



$$(18-(6/3))-2 = 14$$

$18-(6/3-2) = 18$: los paréntesis cambian la agrupación.

$18-6/-2$ se rechaza en B1. Todavía no hay una regla para menos unario.

B2 · Potencias con asociación derecha

```
%left '+' '-'
```

```
%left '*' '/'
```

```
%right '^'
```

```
expr '^' expr
```

```
{ $$ = pow($1, $3); }
```

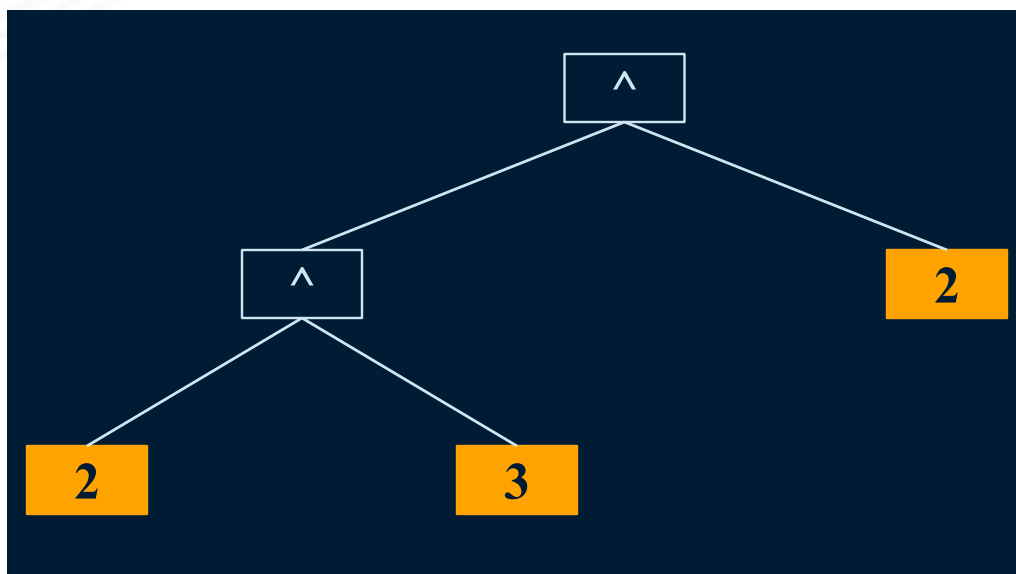
$\wedge$ es potencia en este lenguaje didáctico.

pow requiere `<math.h>` y enlazar la biblioteca matemática cuando la plataforma lo exija.

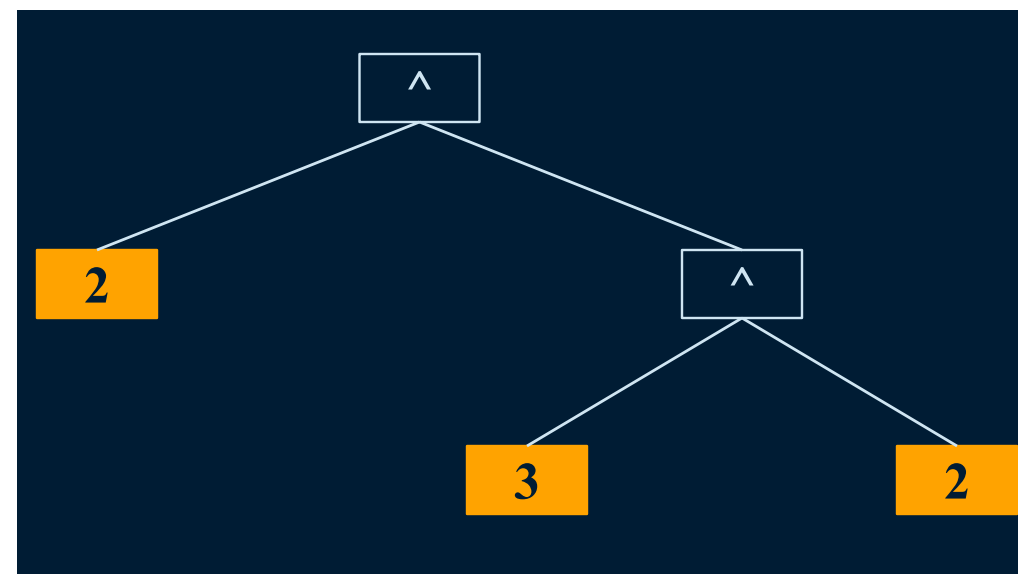
En C, $\$1 \wedge \3 no calcula potencias.

B2 · Potencia encadenada

$$(2^3)^2 = 64$$



$$2^{(3^2)} = 512$$



Con otro ^ como lookahead hay un empate. %right elige shift y agrupa primero la potencia de la derecha.

B2 · Niveles y paréntesis

Entrada	Pasos de evaluación	Resultado
2^3^2	$3^2 = 9$; $2^9 = 512$	512
$2*3^2$	$3^2 = 9$; $2*9 = 18$	18
$(2^3)^2$	$2^3 = 8$; $8^2 = 64$	64

Los paréntesis escritos por el programador siguen controlando la agrupación.

B3 · Escala completa de precedencia

```
%nonassoc '<'
%left '+' '-'
%left '*' '/'
%right UMINUS
%right '^'
```

Precedencia creciente de arriba hacia abajo.

UMINUS es un símbolo ficticio para asignar precedencia a una regla. Flex no lo devuelve.

En Bison moderno puede usarse %precedence UMINUS. Aquí se conserva %right por compatibilidad.

No incluir a la vez %right UMINUS y %precedence UMINUS.

B3 · Precedencia del menos unario

```
| '-' expr %prec UMINUS  
  { $$ = -$2; }
```

```
| expr '<' expr  
  { $$ = ($1 < $3); }
```

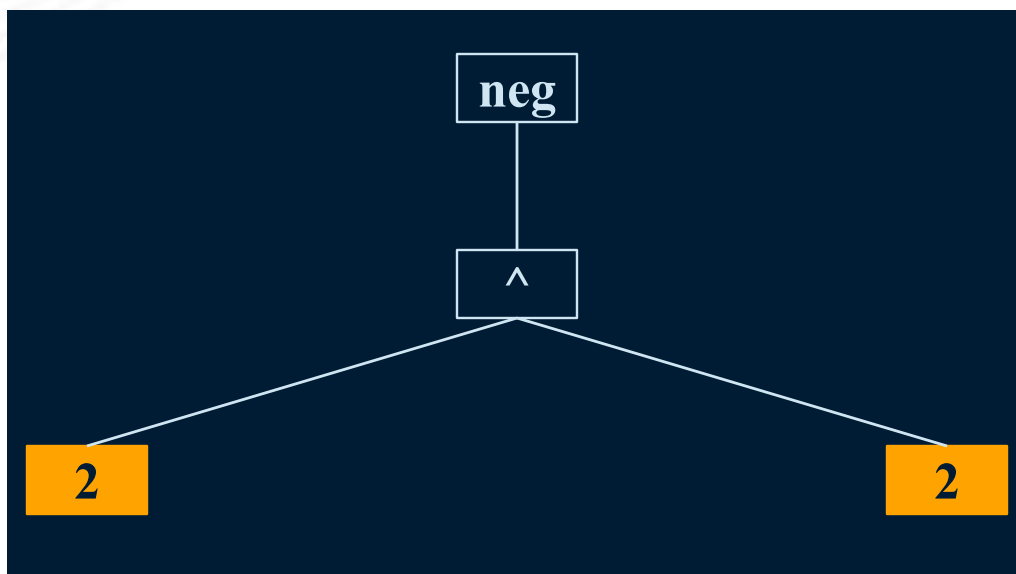
%prec UMINUS cambia la precedencia de esa producción. La resta binaria conserva su nivel.

El operando unario está en \$2. UMINUS no aparece en la entrada.

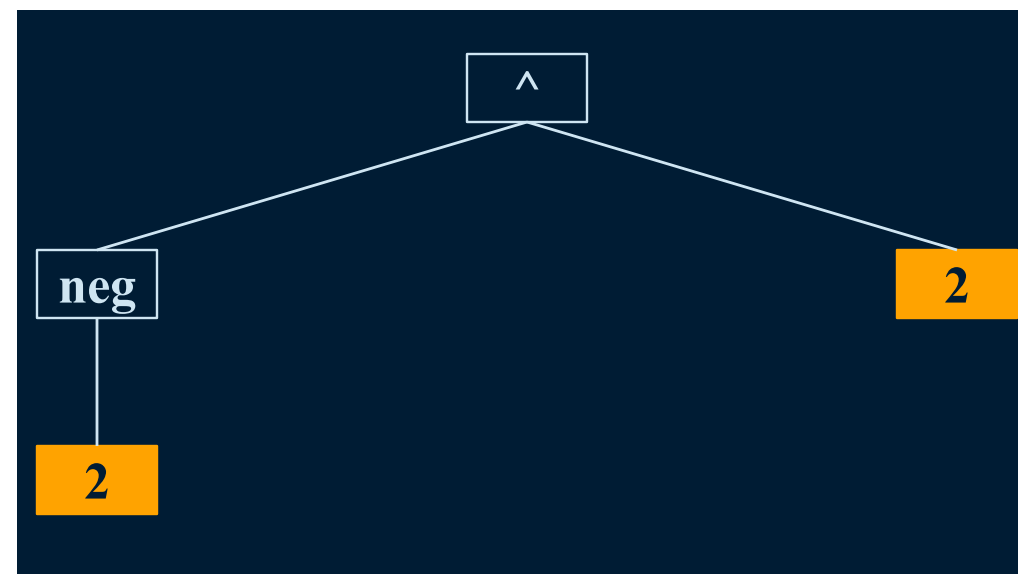
Distinguir el token - de la precedencia asignada a la regla unaria.

B3 · Signo y paréntesis

$$-2^2 = -4$$



$$(-2)^2 = 4$$



^ supera a UMINUS: en -2^2 se calcula primero la potencia.

Los paréntesis de $(-2)^2$ obligan a agrupar el signo con el primer 2.

En este modelo el lexer devuelve - y NUM por separado.

B3 · Expresión integrada

Subexpresión	Resultado
2^2	4
Aplicar menos unario	-4
$2*3$	6
$1+6$	7
Comparar $-4 < 7$	1 (verdadero)

Entrada: $-2^2 < 1+2*3$

Agrupación: $-(2^2) < (1+(2*3))$

Orden didáctico de subexpresiones, no una regla general para todos los lenguajes.

B3 · Rechazo con %nonassoc

$1 < 2 < 3$

$1 < 2$ reconocido
Lookahead: $<$
Empate con %nonassoc

ERROR SINTÁCTICO

$(1 < 2) < 3$

Primero: $1 < 2 = 1$
Después: $1 < 3 = 1$

ACEPTADA

Todos los tokens son reconocibles. El rechazo del encadenamiento es una decisión sintáctica.

%nonassoc restringe lo que acepta el parser respecto de la CFG plana.